Page 1: Introduction to Java

Java Course By CodeWithHarry

Java is an **Object Oriented programming language** developed by **Sun Microsystems** of USA in **1991**.
It was originally called **Oak** by **James Gosling** (one of the inventors of Java!).

JAVA = Purely Object Oriented

How JAVA Works?

Java is **compiled** into the **bytecode** and then it is **interpreted** to machine code.

> **Source Code** —(Compiled)—> **Bytecode** —(Interpreted for a given machine)—> **Machine Code**

JAVA Installation

- Go to Google & type "Install JDK" ⇒ Installs JAVA JDK
- Go to Google & type "Install IntelliJ IDEA" ⇒ Installs JAVA IDE
- **JDK** → **JAVA Development Kit** = Collection of tools used for developing and running Java programs.
- **JRE** → **JAVA Runtime Environment** = Helps in executing programs developed in JAVA.

Page 2: Basic Structure & Naming Conventions

Basic Structure of a Java Program

```java
package com.company; // Groups classes!

public class Main { // Entry point into the application
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Naming Conventions

- **For classes, we use PascalConvention.** First and subsequent characters from a word are capital letters (uppercase).
  - *Example:* `Main`, `MyScanner`, `MyEmployee`, `CodeWithHarry`
- **For functions and variables**, we use **camelCaseConvention**. Here first character is lowercase and the subsequent characters are uppercase.
  - *Example:* `main`, `myScanner`, `myMarks`, `codeWithHarry`

Page 3: Variables and Data Types

Chapter 1 - Variables and Datatypes

Just like we have some rules that we follow to speak English (the grammar), we have some rules to follow while writing a Java program. The set of these rules is called **syntax** (Vocabulary & Grammar of Java).

Variables

A variable is a **container** that stores a value. This value can be changed during the execution of the program.

- *Example:* `int number = 8;`
  - `int` = Data type
  - `number` = Variable name
  - `8` = Value it stores!

Rules for declaring a variable name

1. Must not begin with a **digit** → `int 1harry;` is invalid!
2. Name is **case sensitive** → `harry` and `Harry` are different!
3. Should not be a **keyword** (like `void`).
4. **White Space** not allowed → `int Code With Harry;` is invalid.
5. Can contain alphabets, $ character, _ character, and digits if the other conditions are met.

Page 4: Primitive Data Types

Primitive Data Types

Java is **statically typed** → variables must be declared before use!
There are 8 **primitive data types** supported by Java:

1. **byte**: Value ranges from -128 to 127. Takes 1 byte. Default value is 0.

2. **short**: Value ranges from $-(2^{16})/2$ to $(2^{16})/2 - 1$. Takes 2 bytes. Default value is 0.

3. **int**: Value ranges from $-(2^{32})/2$ to $(2^{32})/2 - 1$. Takes 4 bytes. Default value is 0.

4. **float**: Value ranges from (See Docs). Takes 4 bytes. Default value is 0.0f.

5. **long**: Value ranges from $-(2^{64})/2$ to $(2^{64})/2 - 1$. Takes 8 bytes. Default value is 0.

6. **double**: Value ranges from (See Docs). Takes 8 bytes. Default value is 0.0d.

7. **char**: Value ranges from 0 to 65535 ($2^{16} - 1$). Takes 2 bytes → because it supports unicode. Default value is `\u0000`.

Page 5: Booleans, Literals, and Selection

8. **boolean**: Value can be **true** or **false**. Size depends on JVM. Default value is **false**.

Quick Quiz

Write a Java program to add three numbers.

How to choose data types for our Variables

- **Integral:** `byte`, `short`, `int`, `long(L)`
- **Floats (Decimal):** `float(f or F)`, `double(D or d)`
- **Char:** `(' ')`
- **Boolean:** `(true/false)`

Literals

A constant value which can be assigned to the variable is called as a **literal.**

- `101` → Integer literal
- `10.1f` → Float literal
- `10.1` → Double literal (default type for decimals)
- `'A'` → Character literal
- `true` → Boolean literal
- `"Harry"` → String literal

Page 6: Keywords & Reading Input

Keywords

Words which are reserved and used by the Java Compiler. They cannot be used as an Identifier.

Reading data from the Keyboard

In order to read data from the keyboard, Java has a **Scanner class.** Scanner class has a lot of methods to read the data from the keyboard.

- `Scanner s = new Scanner(System.in);` → Read from keyboard
- `int a = s.nextInt();` → Method to read from the keyboard (Integer in this case)

Exercise 1.1

Write a program to calculate percentage of a given student in CBSE board exam. His marks from 5 subjects must be taken as input from the keyboard. (Marks are out of 100).

Page 7: Chapter 1 Practice Set

1. Write a program to sum three numbers in Java.
2. Write a program to calculate CGPA using marks of three subjects (out of 100).
3. Write a Java program which asks the user to enter his/her name and greets them with "Hello <name>, have a good day" text.
4. Write a Java program to convert Kilometers to miles.
5. Write a Java program to detect whether a number entered by the user is integer or not.

Page 8: Operators and Expressions

Chapter 2 - Operators and Expressions

Operators are used to perform operations on variables and values.

7 (operand) + (operator) 11 (operand) = 18 (Result)

Types of operators

- **Arithmetic Operators:** +, -, *, /, %, ++, --
- **Assignment Operators:** =, +=
- **Comparison Operators:** ==, >=, <=
- **Logical Operators:** &&, ||, !
- **Bitwise Operators:** &, | (Operates bitwise)

Arithmetic operators cannot work with booleans. % operator can work on floats & doubles.

Precedence of operators

The operators are applied and evaluated based on precedence. For example `(+, -)` has less precedence compared to `(*, /)`. Hence * & / are evaluated first. In case we like to change this order, we use **parenthesis**.

Associativity

Associativity tells the direction of execution of operators.

- `*` `/` → L to R
- `+` `-` → L to R
- `++`, `=` → R to L

Page 9: Arithmetic Operations & Increment

Quick Quiz

How will you write the following expressions in Java?

1. $$\frac{x-y}{2}$$
2. $$\frac{b^2-4ac}{2a}$$
3. $$v^2-u^2$$
4. $$a*b-d$$

Resulting data type after arithmetic operation

- `byte + short` → `int`
- `short + int` → `int`
- `long + float` → `float`
- `int + float` → `float`
- `char + int` → `int`
- `char + short` → `int`
- `long + double` → `double`
- `float + double` → `double`

Increment and Decrement Operators

- a++, ++a → Increment operators (Data type remains same)
- a--, --a → Decrement operators

Quick Quiz

- a++ → first use the value and then increment
- ++a → first increment the value then use it

Page 10: Quiz Result

Quick Quiz

What will be the value of the following expression (x)?

java

```java
int y = 7;
int x = ++y * 8;
// Value of x? (Result would be 64)
```

java

```java
int y = 7;
int x = ++y * 8;
// Value of x? (Result would be 64)
```

java

```java
int y = 7;
int x = ++y * 8;
// Value of x? (Result would be 64)
```


```java
char a = 'B';
a++; // a is now 'C'
```

Chapter 2 - Practice Set

1. What will be the result of the following expression

```
float a = 7 / 4 * 9 / 2
```

2. Write a Java program to encrypt a grade by adding 8 to it. Decrypt it to show the correct grade.

3. Use comparison operators to find out whether a given number is greater than the user entered number or not.

4. Write the following expression in a Java program:

$$\frac{v^2 - u^2}{2as}$$

5. Find the value of the following expression:

```
int x = 7
int a = 7 * 49 / 7 + 35 / 7
```

Value of a?

Chapter 3 – Strings

A string is a sequence of characters.
A string is instantiated as follows:

```
String name;
name = new String("Harry");
```

String is a class but can be used like a data type.
(Strings are immutable and cannot be changed)

```
String name = "Harry";
```

Different Ways to Print in Java

We can use the following ways to print in Java:

1. `System.out.print()` → No newline at the end
2. `System.out.println()` → Prints a new line at the end
3. `System.out.printf()`
4. `System.out.format()`

Example:

```
System.out.printf("%c", ch);
```

Format specifiers:

- `%d` → for int
- `%f` → for float
- `%c` → for char
- `%s` → for string

String Methods

String methods operate on Java strings. They can be used to find the length of the string, convert to lowercase, etc.

Commonly Used String Methods:

```
String name = "Harry";
```

1. `name.length()`
   → Returns length of string name (5 in this case)
2. `name.toLowerCase()`
   → Returns a new string with all lowercase characters
3. `name.toUpperCase()`
   → Returns a new string with all uppercase characters
4. `name.trim()`
   → Returns a new string after removing leading and trailing spaces
5. `name.substring(int start)`
   → Returns substring from start index to end
   Example: `substring(3)` returns `"ry"`
   (Note: index starts from 0)
6. `name.substring(int start, int end)`
   → Returns substring from start index to end index
   (start included, end excluded)
7. `name.replace('r', 'p')`
   → Returns a new string after replacing characters
   Example: `"Happy"`
8. `name.startsWith("Ha")`
   → Returns true if string starts with "Ha"
9. `name.endsWith("ry")`
   → Returns true if string ends with "ry"
10. `name.charAt(2)`
    → Returns character at given index (r in this case)
11. `name.indexOf("ar")`
    → Returns index of first occurrence (1 in this case), otherwise -1
12. `name.indexOf("s", 3)`
    → Searches from index 3, returns index or -1
13. `name.lastIndexOf("r")`
    → Returns last index of character (3 in this case)
14. `name.lastIndexOf("r", 2)`
    → Returns last index before index 2

16. `name.equalsIgnoreCase("harry")`
    → **Returns true if equal ignoring case**

Escape Sequence Characters

Sequence of characters after backslash \ are called escape sequence characters.

They consist of more than one character but represent a single character in a string.

Examples:

- `\n` → **newline**
- `\t` → **tab**
- `\'` → **single quote**
- `\\` → **backslash**

Chapter 3 – Practice Set

1. **Write a Java program to convert a string to lowercase.**
2. **Write a Java program to replace spaces with underscores.**
3. **Write a Java program to fill in a letter template:**

```
letter = "Dear <|name|>, Thanks a lot"
```
